

DeText: A Deep Text Ranking Framework with BERT

자연어 처리팀

김은희, 백지윤, 주정현, 진명훈

A close-up, low-angle shot of a chocolate cake with white frosting and a blue candle. The cake is in the foreground, and the background is a blurred city street at night with warm lights and buildings.

Introduction

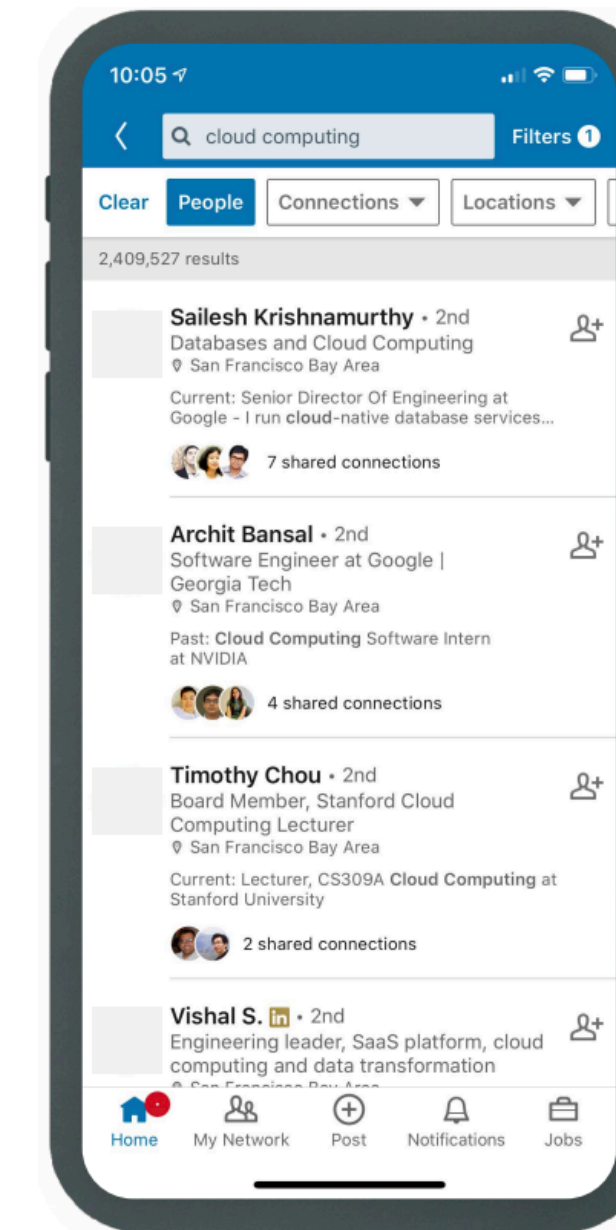
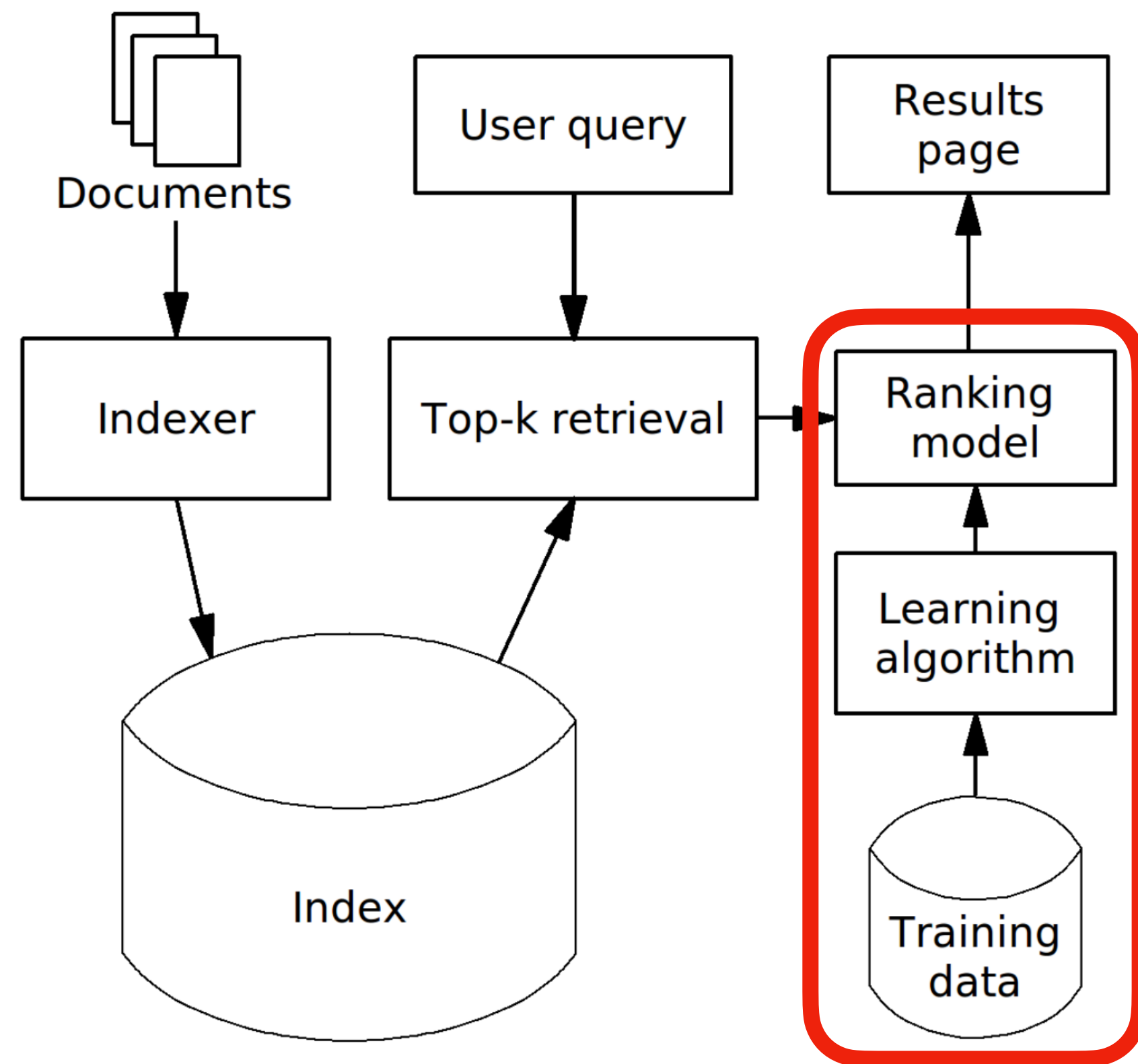
Background

DeText Framework with BERT

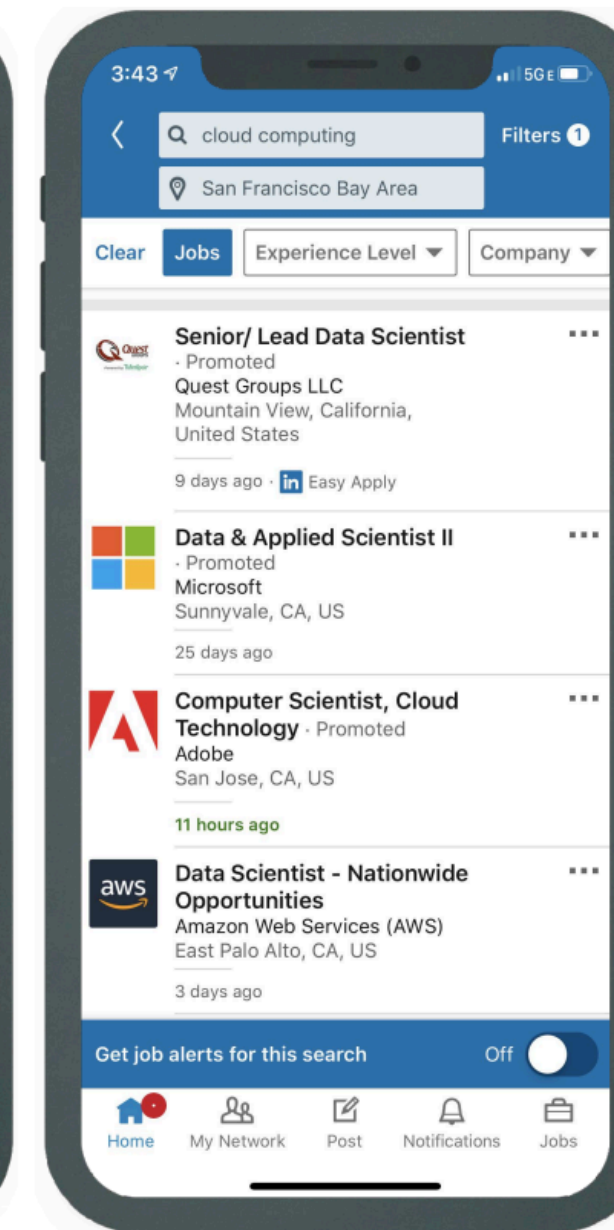
Experiments

Application

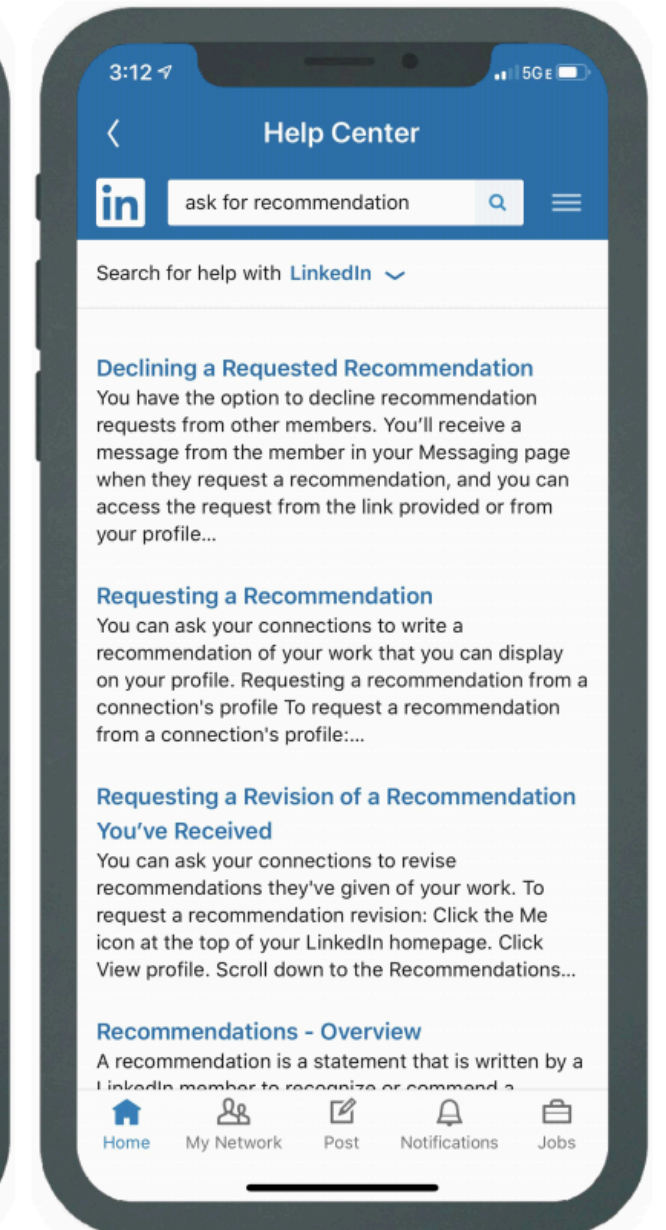
Introduction



(a) People Search



(b) Job Search



(c) Help Center

“Ranking is the most important component in a search system”

“Traditional ranking approaches rely on word/phrase exact matching”

“To enhance contextual modeling, contextual embedding such as BERT has been proposed”

“Heavy BERT computation on the fly”

“Interaction based structure(query and document) precludes embedding pre-computing ”

A close-up, low-angle shot of a chocolate cake with white frosting and a blue candle. The cake is in the foreground, and the background is a blurred city street at night with buildings and lights.

Introduction

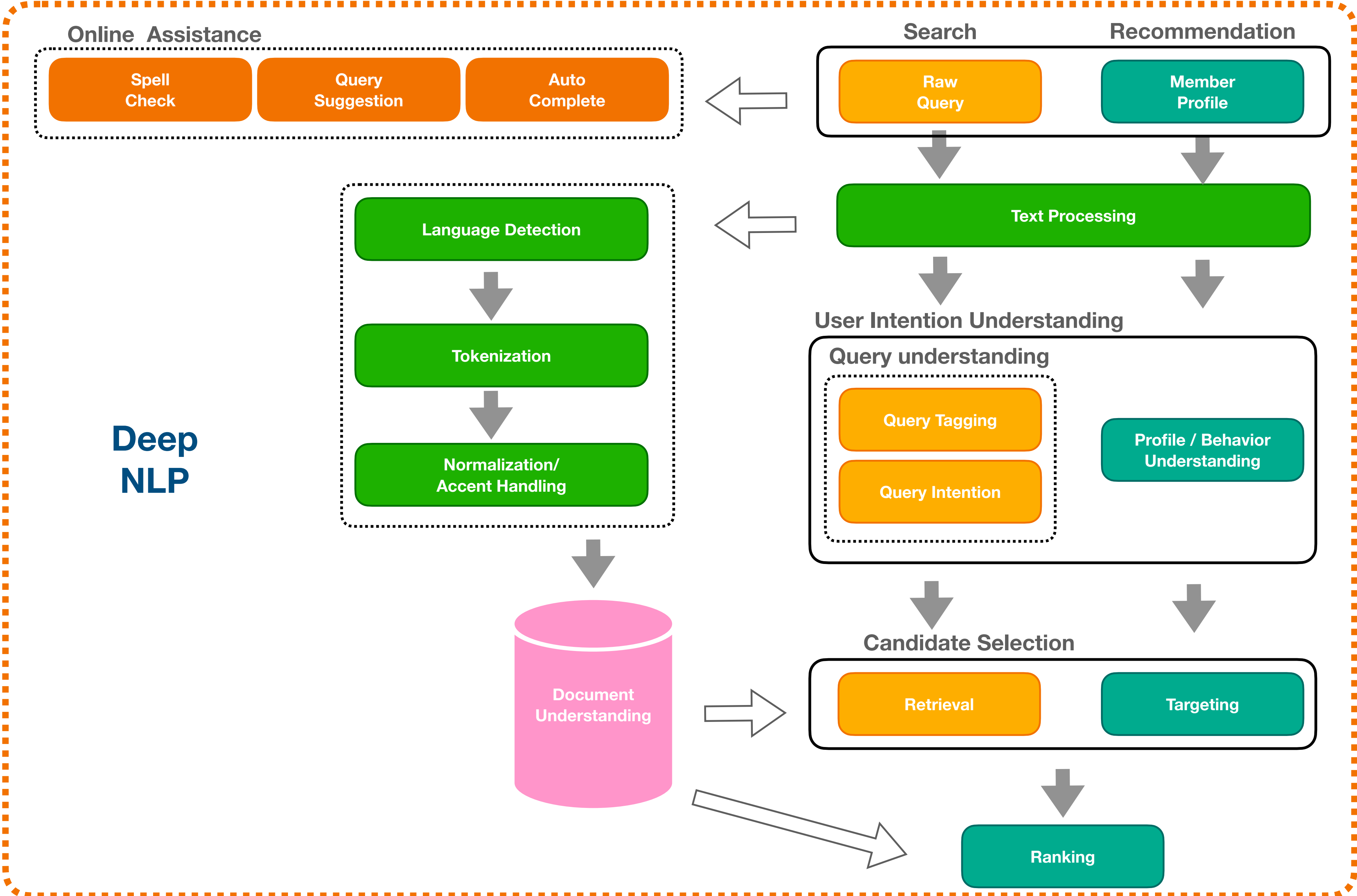
Background

DeText Framework with BERT

Experiments

Application

Search and Recommendation Ecosystem (LinkedIn)



Deep NLP based Ranking Models (Related Work)

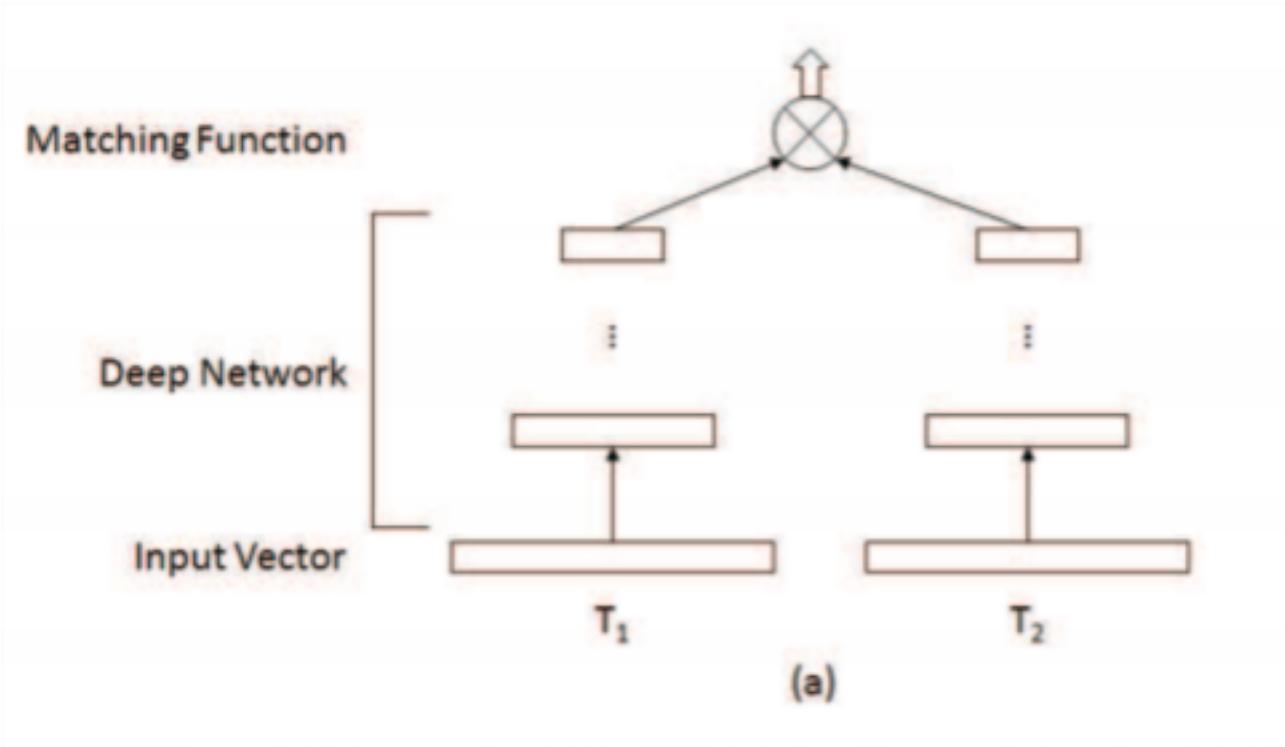
랭킹 시스템에서 일반적인 딥러닝 모델은 두 Text의 매칭 문제로 보는데, 접근법에 따라 두가지로 나눔

representation-focused

각 Text의 의미를 잘 표현하는 구조에 집중하여
Query, Document의 의미 매칭에 집중

검색어와 검색 결과를 각각 input 으로 모델에 입력 -> 대칭적 아키텍처

DSSM, ARC-I

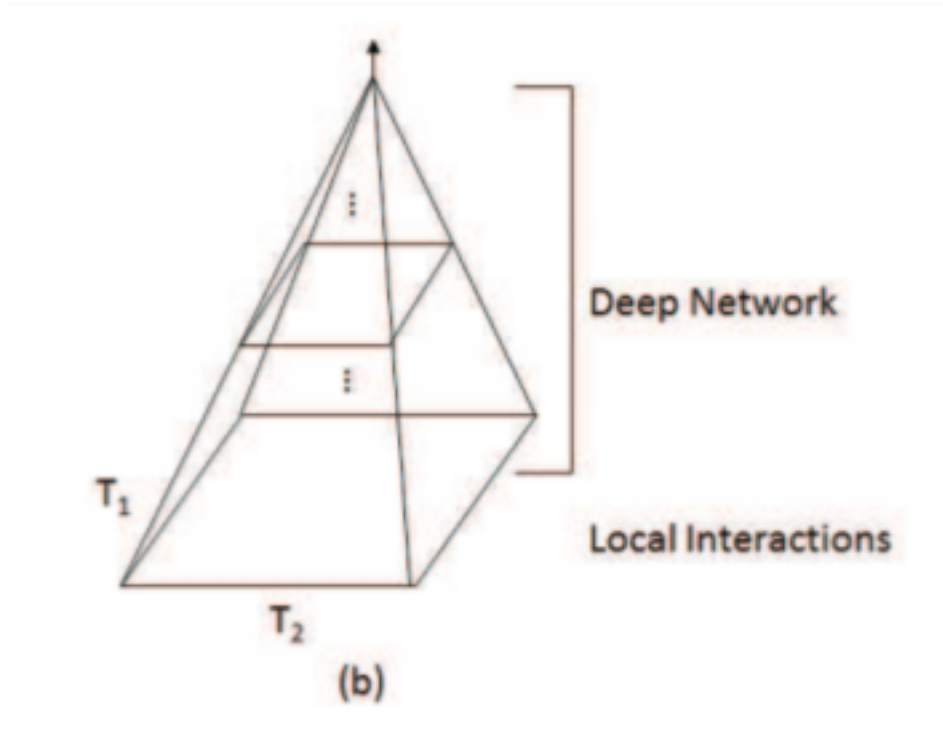


Interaction-focused

두 Text의 상호작용을 계층적으로 배워
관련성을 매칭 시키는데 집중

검색어와 검색 결과의 연산을 거쳐 계층적인 일치도를 파악 -> 피라미드모양의 아키텍처

Deep Match, ARC-II



성능

<

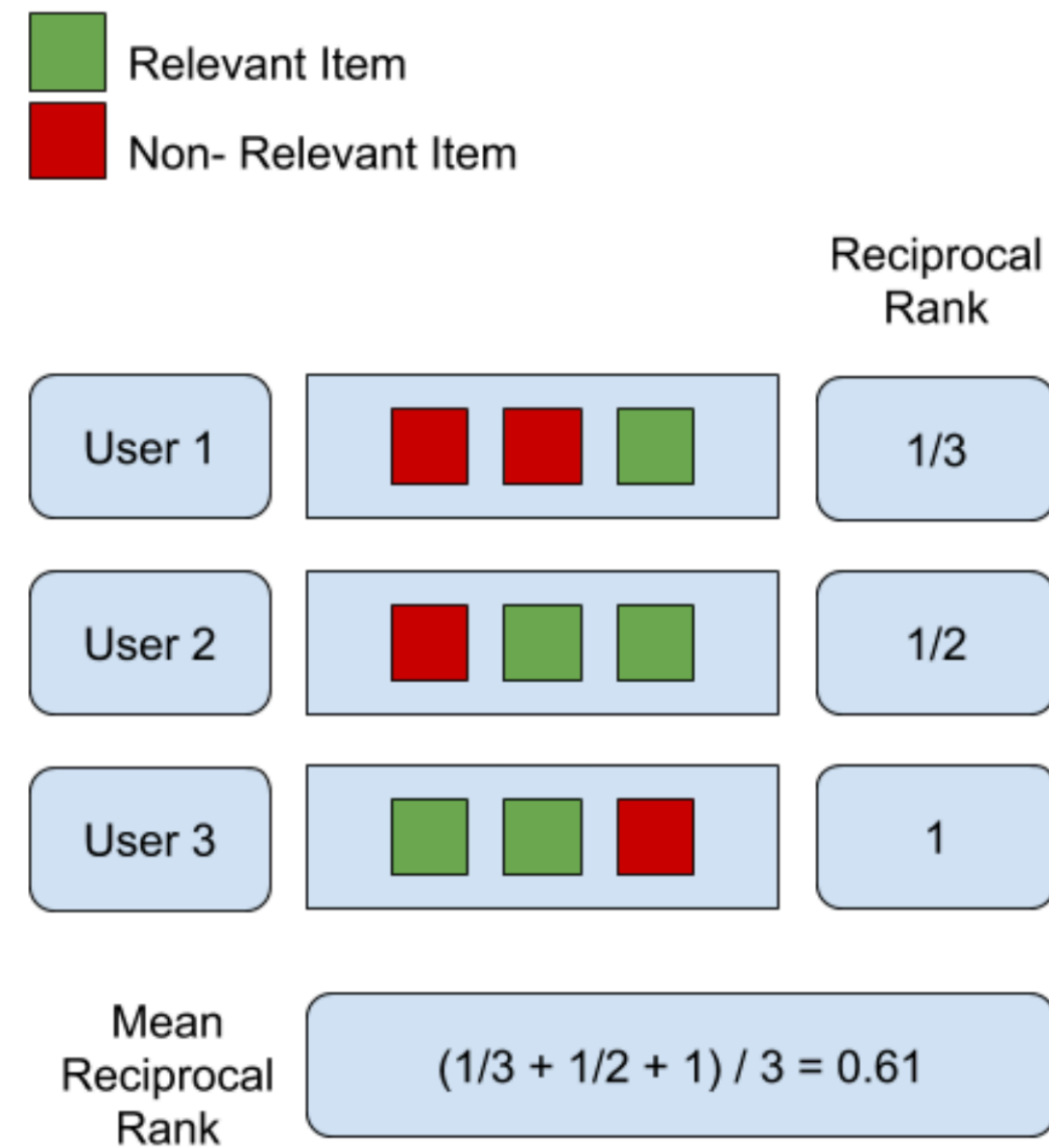
속도

>

BERT 는 query string과 document string이 하나의 문장으로 연결되기 때문에 interaction-focused로 분류
모든 단어의 pariwise 비교로 시간이 오래 걸림

LTR(Learning To Rank) Metric

MRR(Mean Reciprocal Rank)

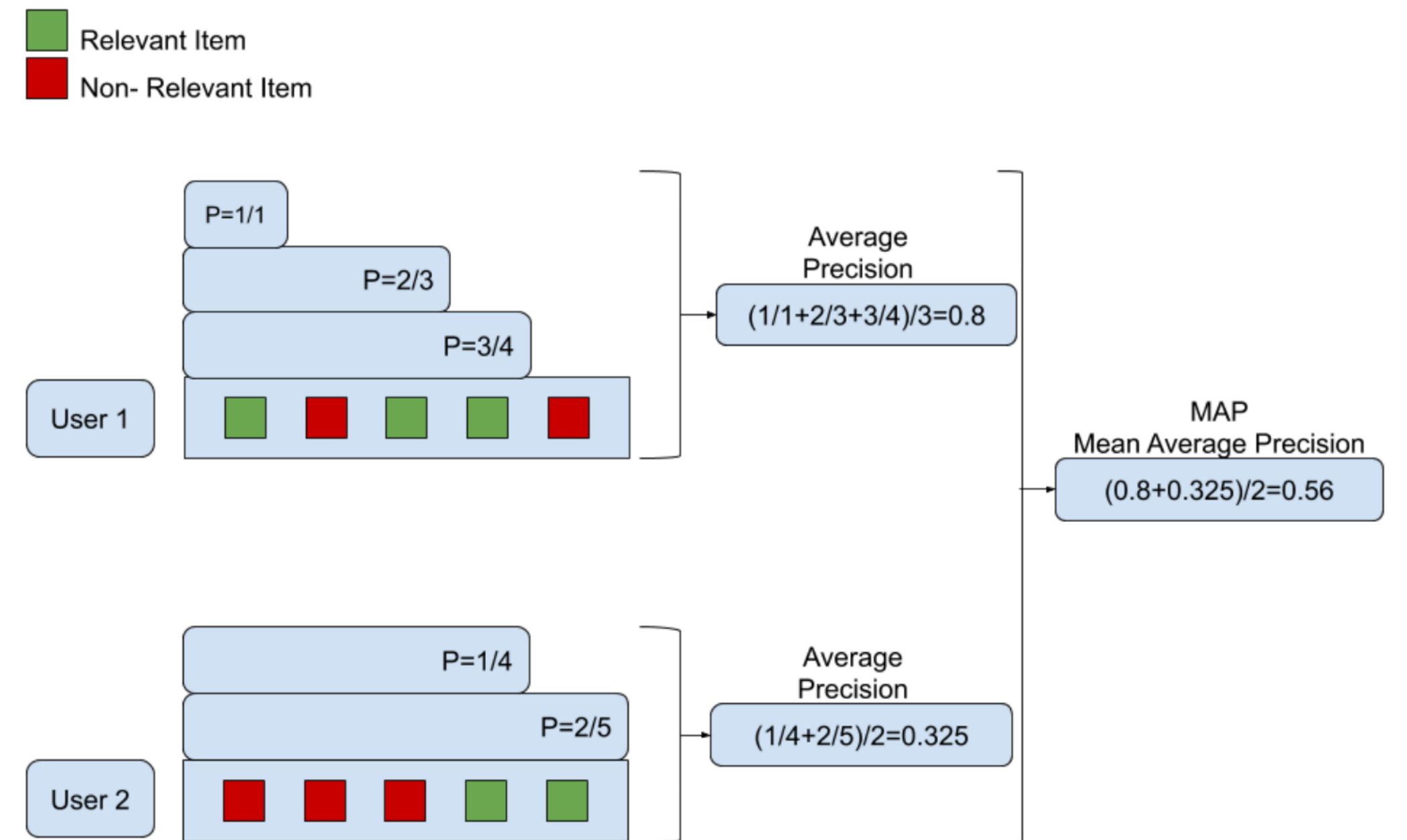


가장 상위의 관련 문서의 위치만을 고려
간단하다

사용자에게 가장 관련있는 콘텐츠가 상위에 위치 되어 있는가를 평가할때 용이

관련 문서의 갯수가 달라도 첫번째 관련 문서의 위치가 같으면 같은 점수
2,3번째 관련 문서는 평가하지 않음

MAP(Mean Average Precision)



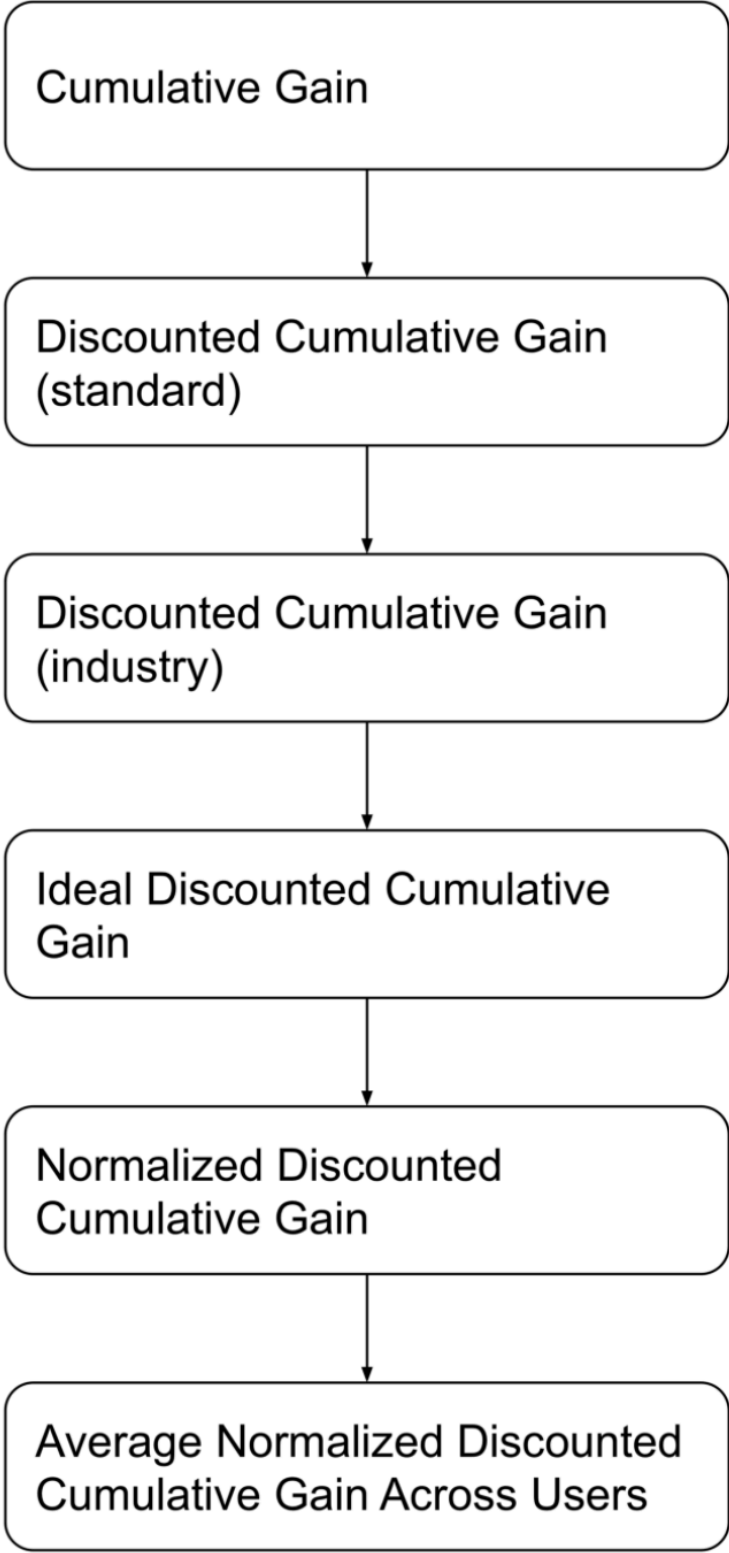
관련 문서의 위치까지의 목록에 대해서 precision을 계산
사용자별로 평균내고 그 결과를 다시 평균

우선순위를 고려한 성능 평가가 가능
문서의 관련 여부를 binary로 나눌수 있으면 좋은 평가 모델
사용자에게 더 관련성 있는 문서를 상위로노출 하기위한 평가로서는 부족

LTR(Learning To Rank) Metric

NDCG(Normalized Discounted Cumulative Gain)

더 관련 있는 결과를 상위에 노출 시키는지에 대한 평가



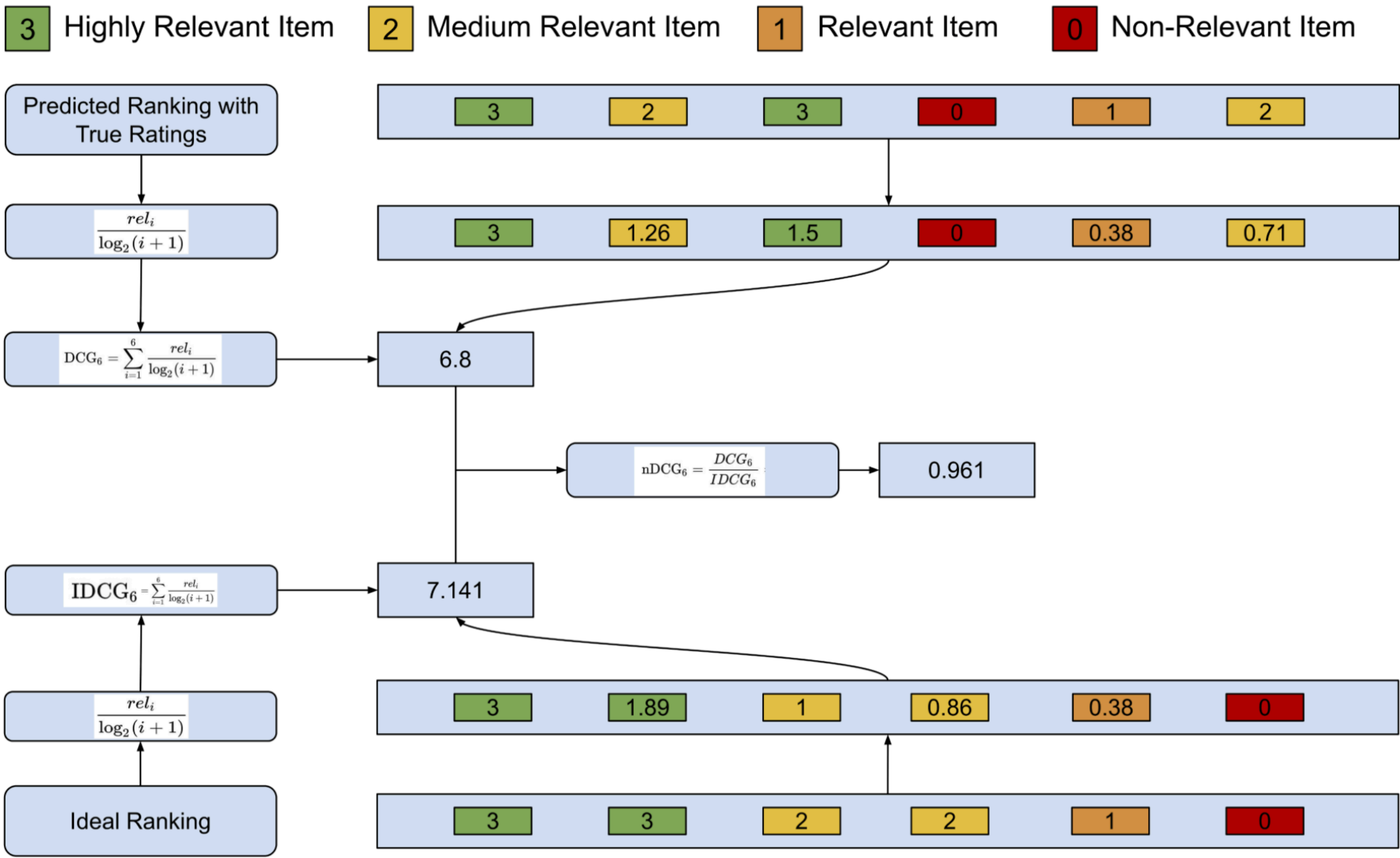
$CG_p = \sum_{i=1}^p rel_i$ p elements in the recommended ranking
graded relevance of the result at position i (gain)

$DCG_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)}$ logarithmic reduction factor to penalize proportionally to the position of the result

$DCG_p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log_2(i+1)}$ graded relevance to get a stronger emphasis on retrieving relevant documents

$IDCG_p = \sum_{i=1}^{|REL_p|} \frac{2^{rel_i} - 1}{\log_2(i+1)}$ list of relevant documents (ordered by their relevance) in the corpus up to position p

$nDCG_p = \frac{DCG_p}{IDCG_p}$



LTR Loss Function (Pointwise, Pairwise, Listwise)

- 기존의 TF-IDF, BM25 등 키워드-문서 사이의 관련도를 잘 모델링 해서 검색 품질에 집중 했다면
- LTR(Learning to Rank)는 문서를 “어떻게 더 잘 정렬할 것인가?”
- 구체적인 score 보다는 list order를 학습

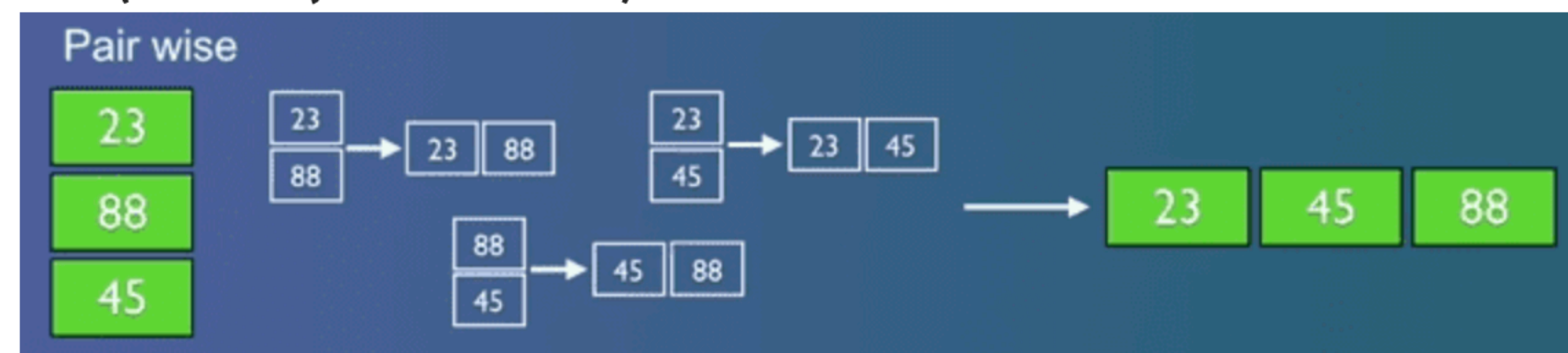
Pointwise

하나의 아이템 단위의 접근, ranking을 prediction 한다는 관점과는 맞지 않음



Pairwise

아이템을 쌍 으로 활용, 랭킹의 ground truth와 가장 많은 pair가 일치하는 순서를 찾아냄. pair의 조합 최적화가 중요



Listwise

Q에 반환된 문서 리스트 전체를 ground truth와 비교, 랭킹 순위를 가장 잘 나열하는게 목적이어서 복잡도가 높고, 결과가 좋음



A close-up, low-angle shot of a chocolate cake with white frosting and a blue candle. The cake is in the foreground, and the background is a blurred city street at night with warm lights and buildings.

Introduction

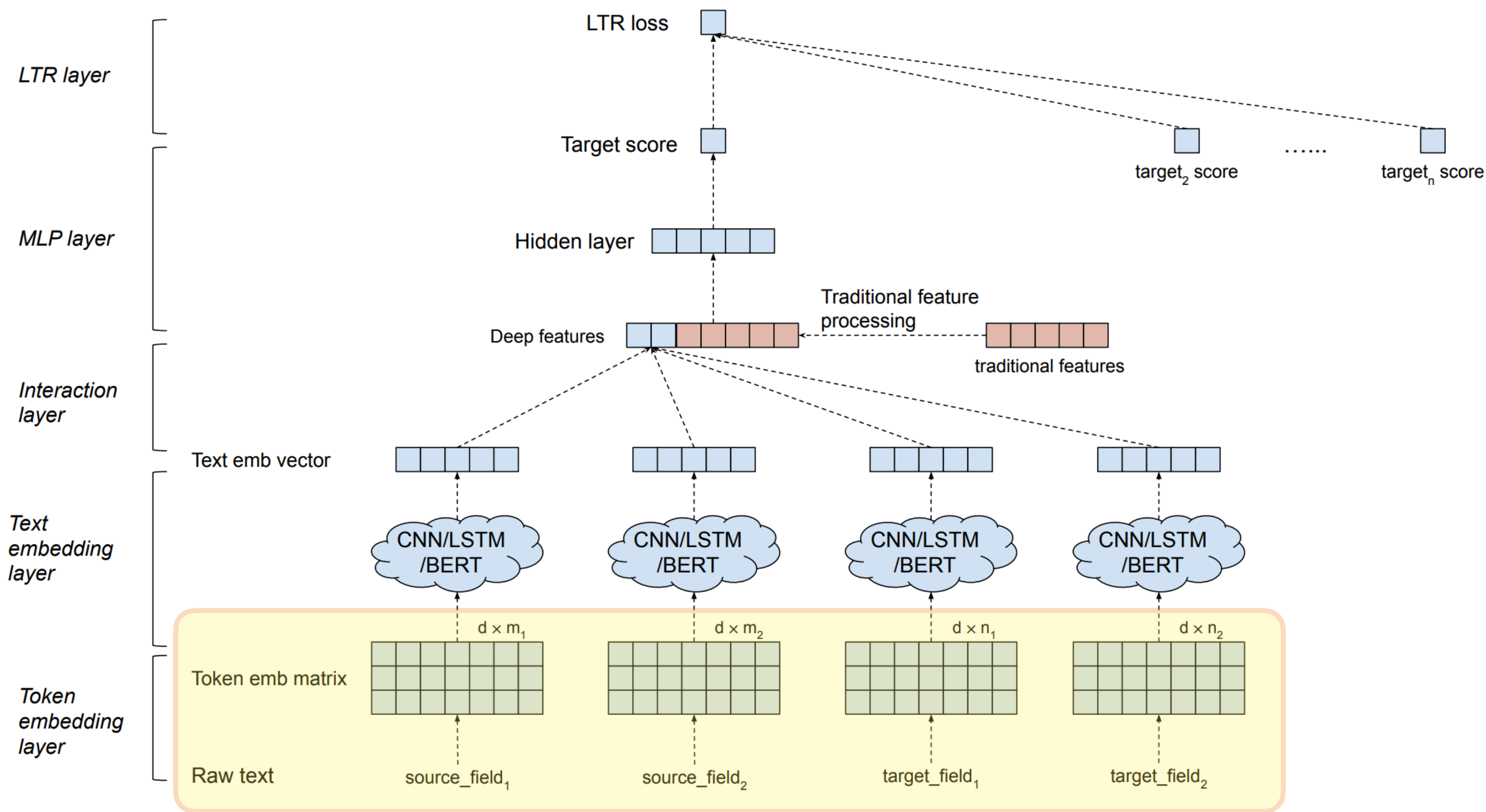
Background

DeText Framework with BERT

Experiments

Application

DeText Framework Design

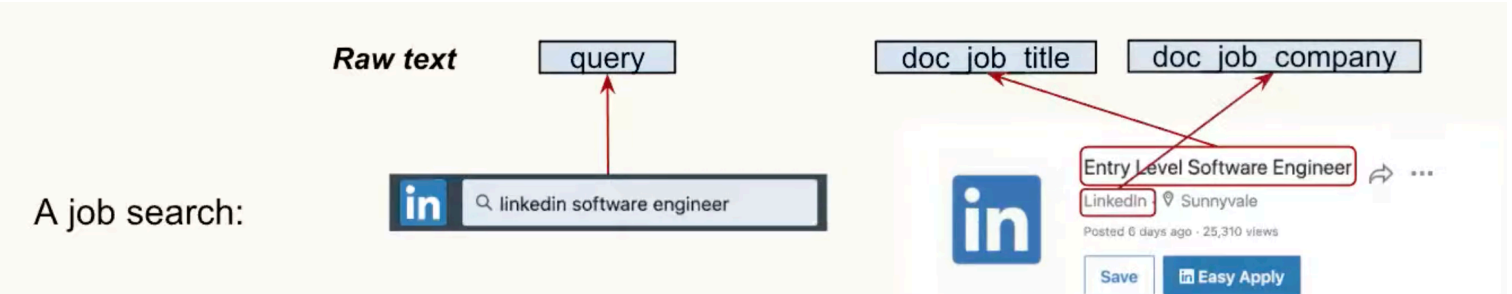


Input Text Data

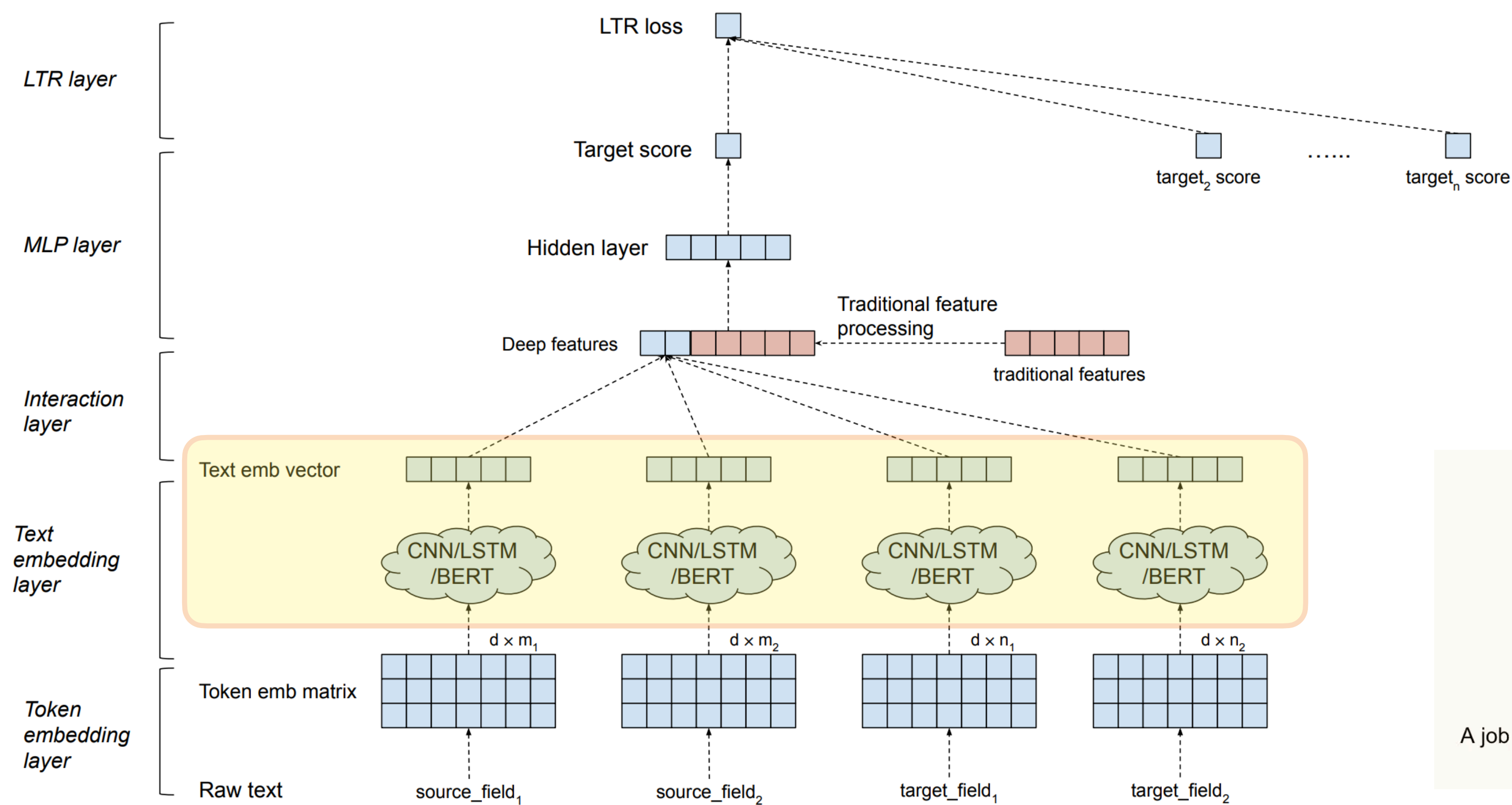
- Source 는 queries or user profiles
- Target은 documents
- Source, Target 에 여러 Field 가 있을 수 있음
- Better and more robust results

Token Embedding Layer

- d x m , m tokens, d token embedding dimensions
- CNN/LSTM : words
- BERT : subwords

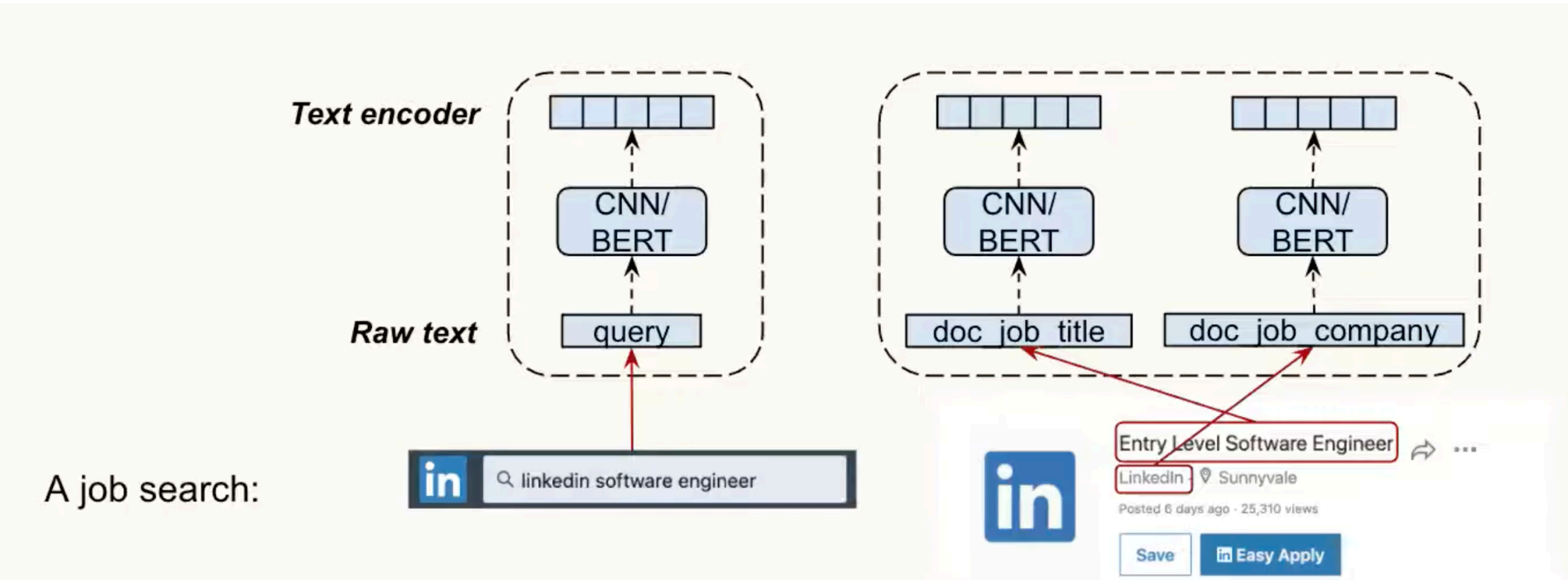


DeText Framework Design

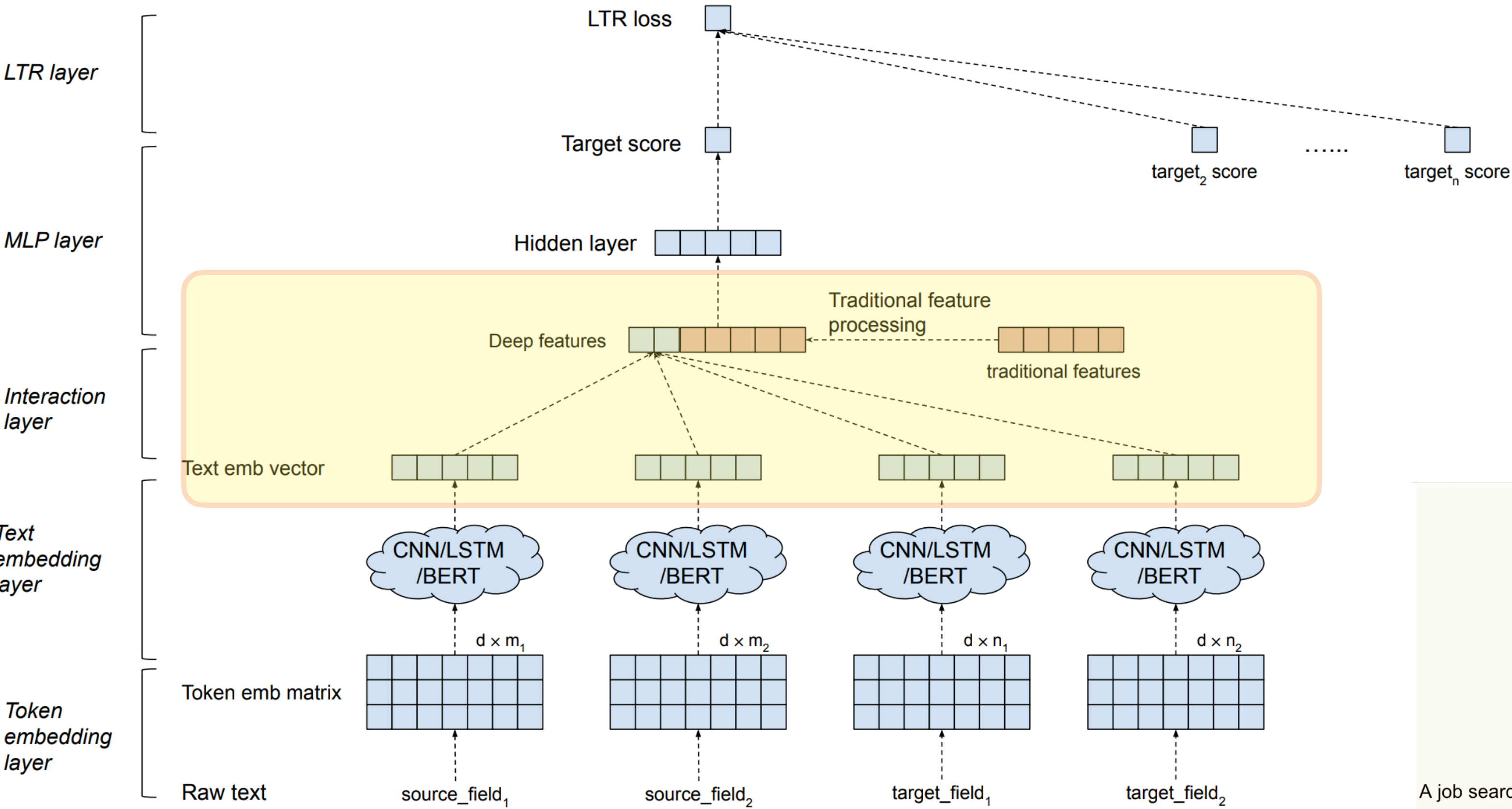


Text Embedding Layer

- Representation base model 구조에서 각 필드마다 독립 임베딩
- BERT, CNN, LSTM 등 다양한 딥러닝 모델을 사용 가능



DeText Framework Design



Interaction Layer

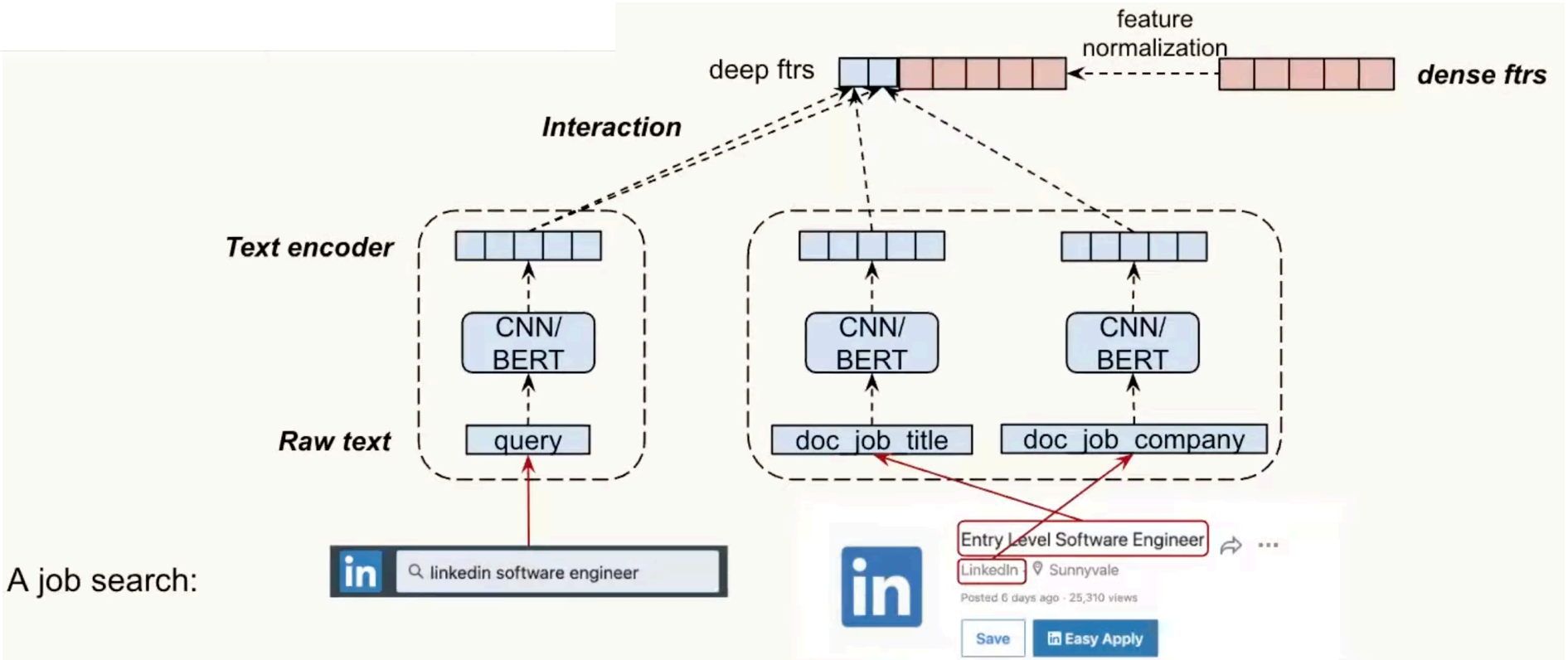
- Source와 target의 Text embedding 이 생성 후 발생
- 3개의 Interaction methods
(cosign similarity, Hadamard product, Concatenation)

Table 2: Interaction features.

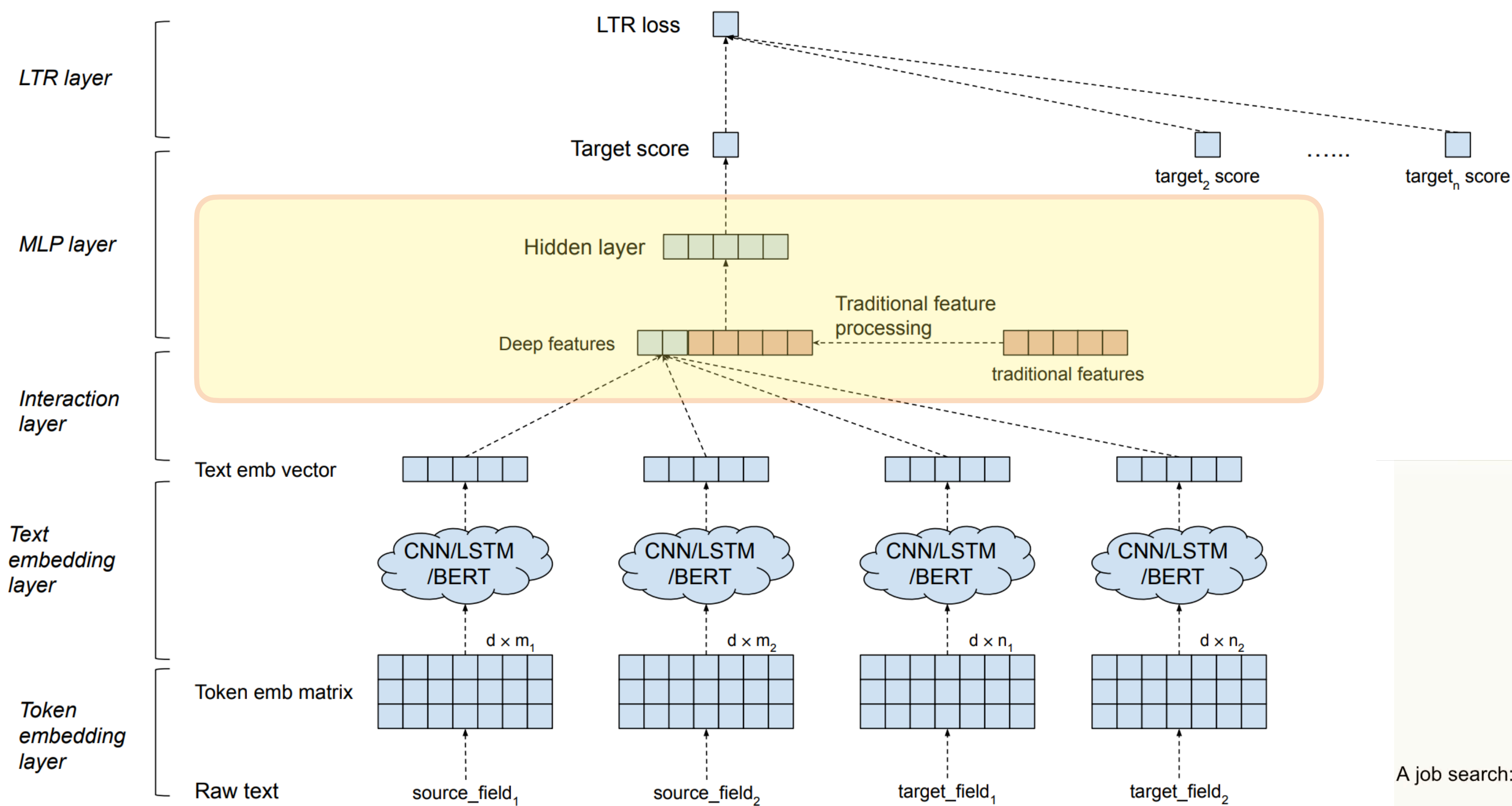
Cosine similarity	$\frac{u_s^T u_t}{\ u_s\ \cdot \ u_t\ }$	one feature per source/target pair
Hadamard product	$u_q \cdot u_d$	d features per source/target pair
Concatenation	$u_q \oplus u_d$	d features per text field

Traditional Feature Processing

- Hand-crafted features
- Personalization feature
- Social networks features
- User behavior feature

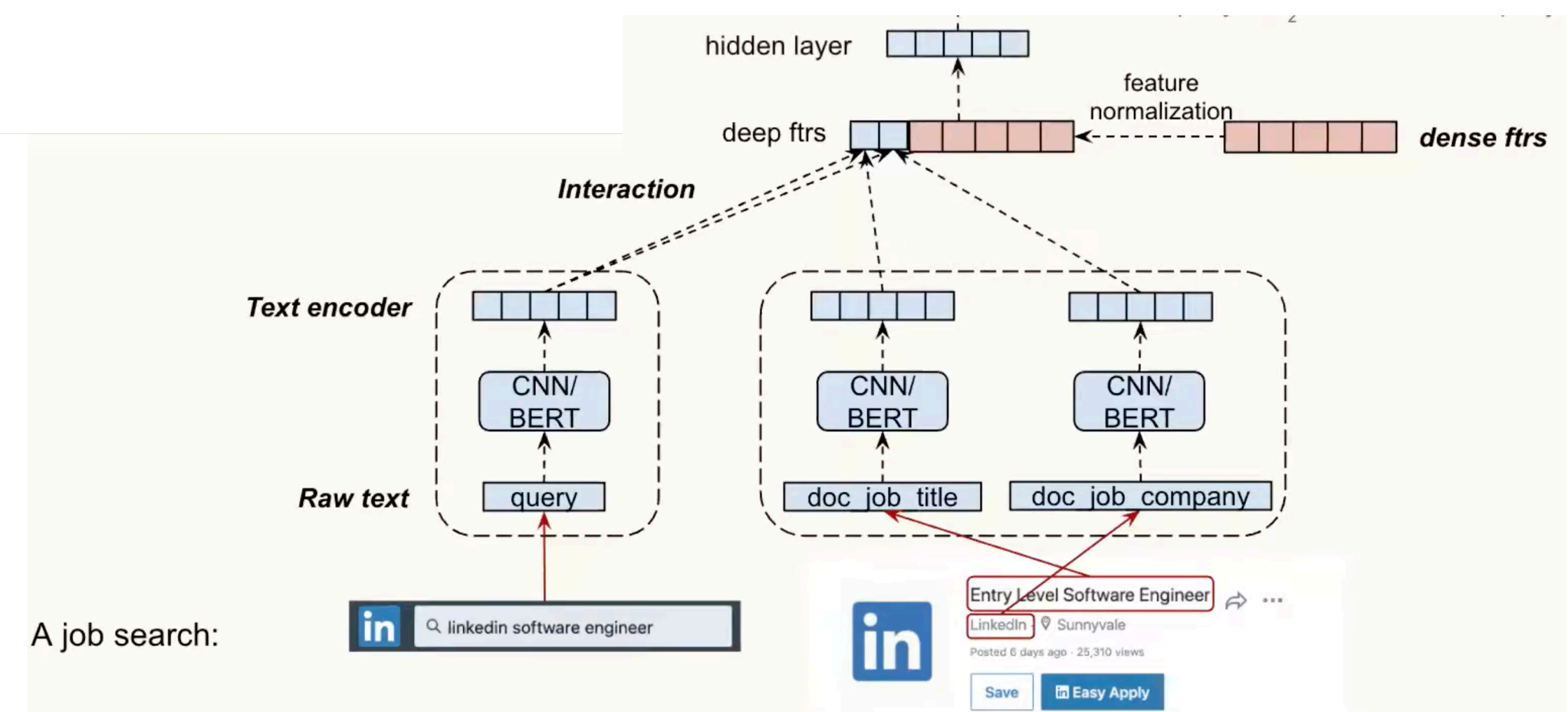


DeText Framework Design

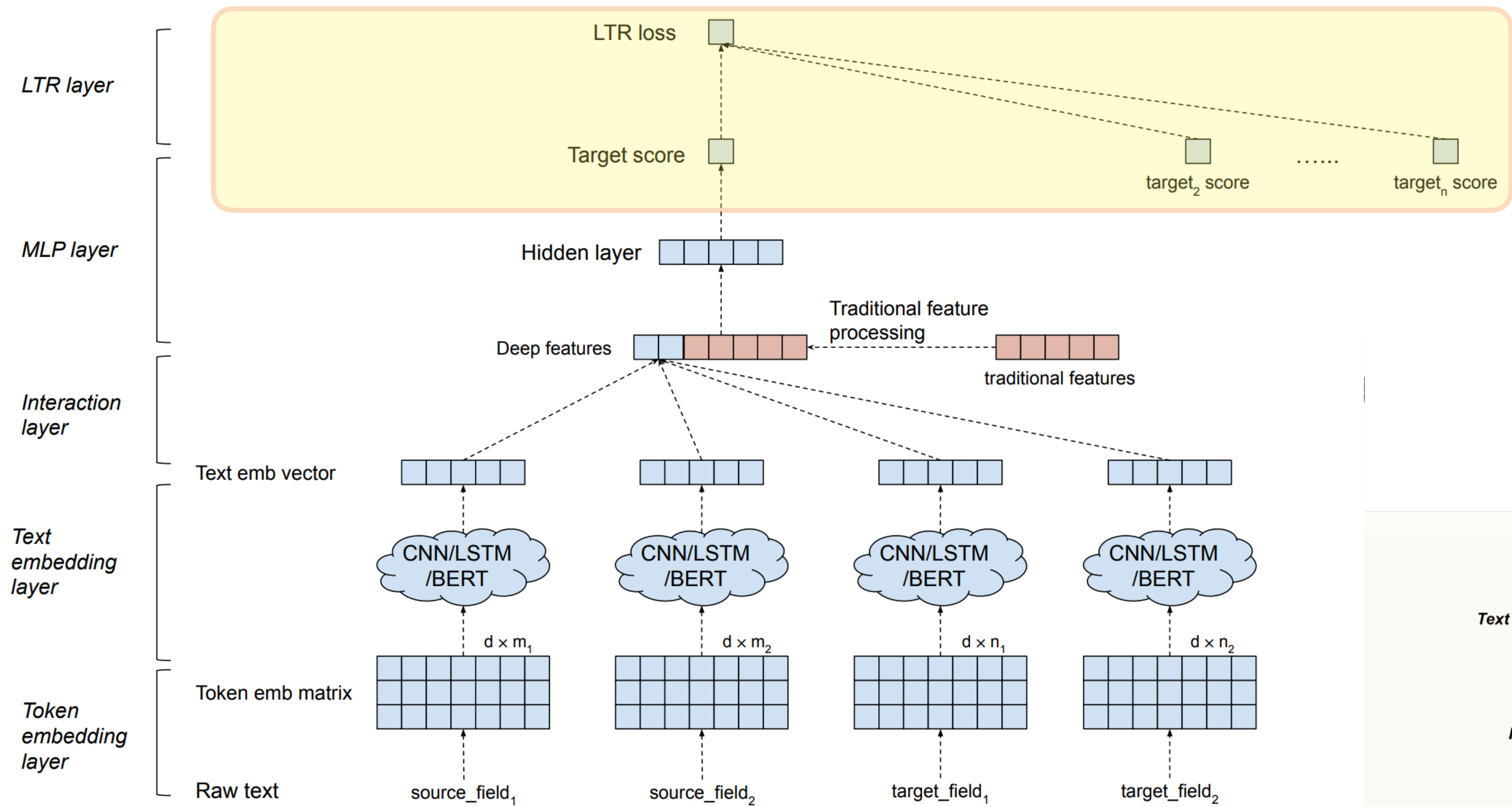


MLP(Multilayer Perceptron) Layer

Deep features와 traditional features 간의 연관성을 추출

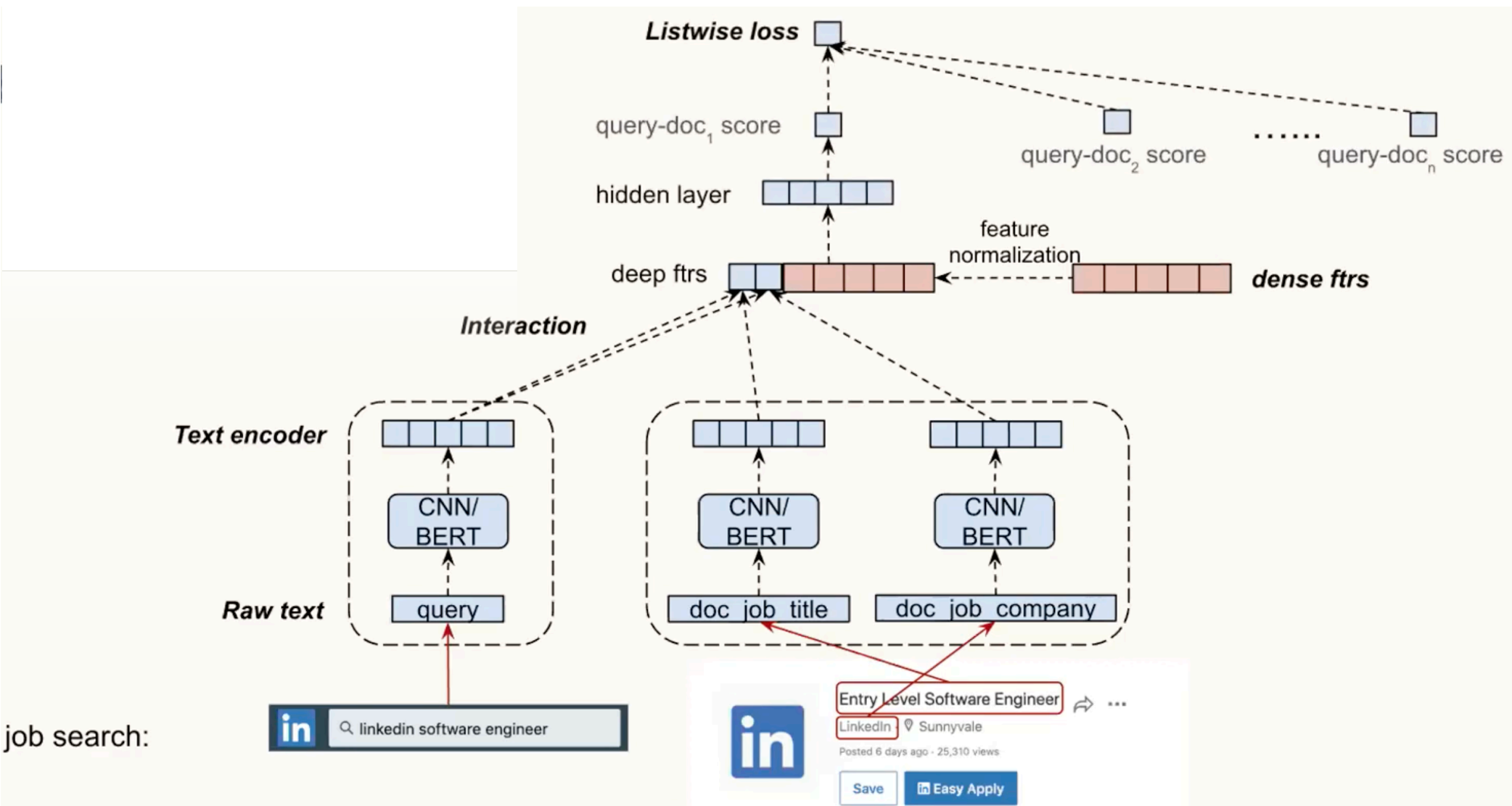


DeText Framework Design

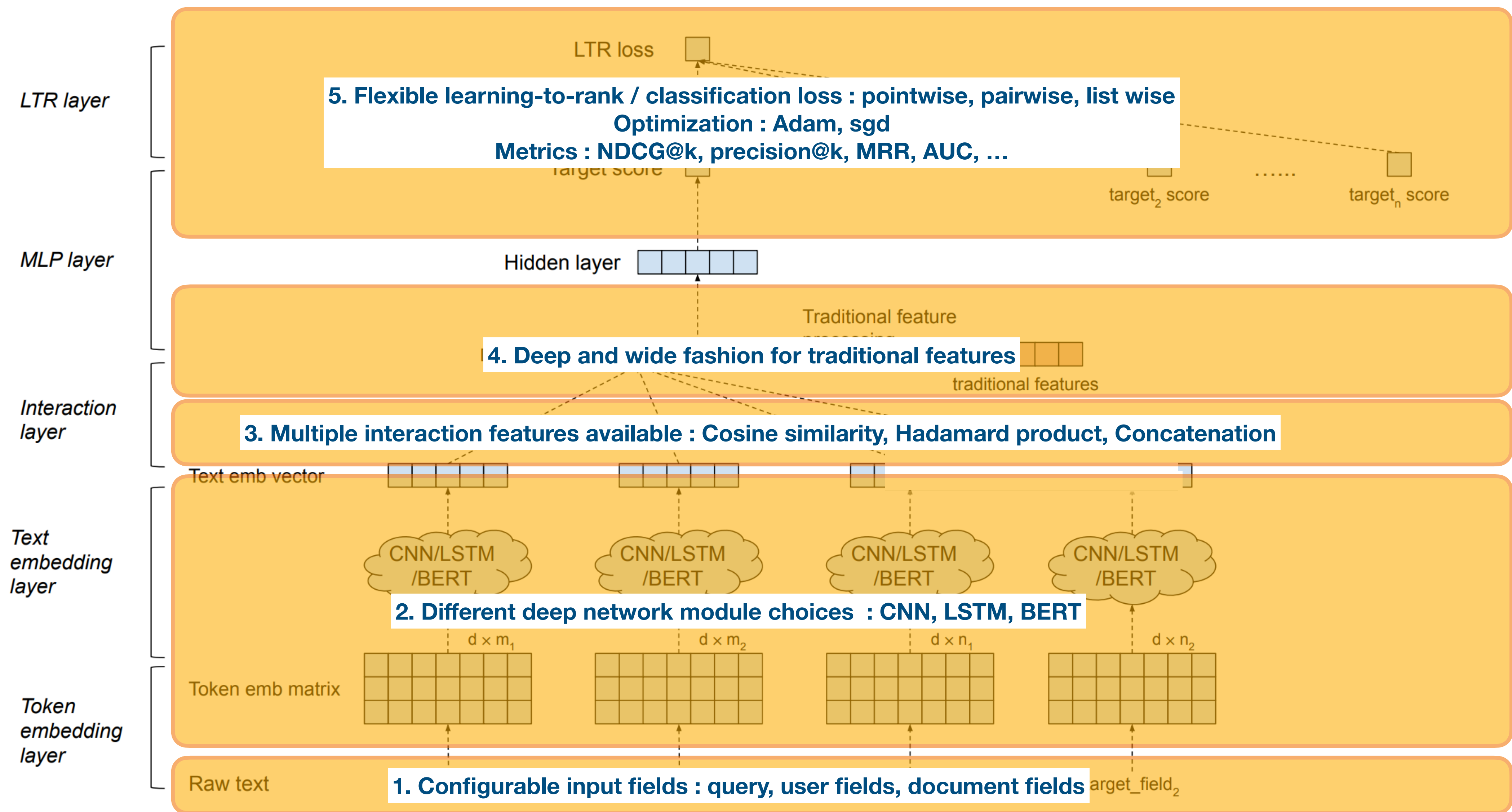


LTR(Learning to Rank) Layer

- Pointwise, Pairwise, Listwise 를 제공
- Click probability 이 중요한 모델인 pointwise 사용
- 상대적인 위치가 중요하면 pairwise/listwise 사용



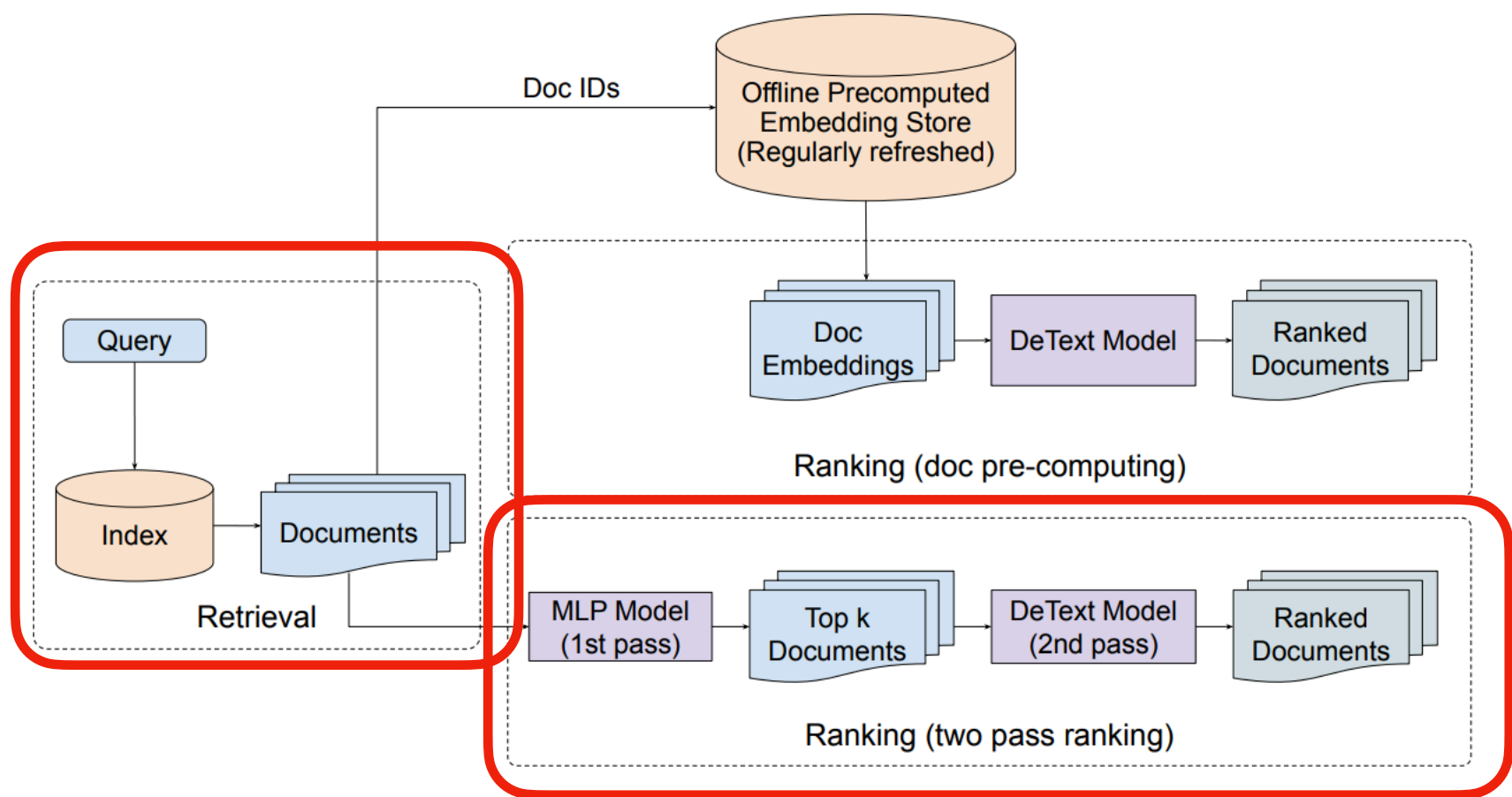
DeText Framework Design



Online Deployment

General Challenge

- * computation grows linearly with # of documents
- Challenge for BERT serving
- * 12-layer Google BERT has 110 million parameters
 - * Could be 30x times slower than CNN



Two pass ranking

- 1 step : MLP model uses traditional features only
 - 2 step : Top k ranked documents are sent to DeText model
-
- Benefit
 - Easy to implement and deploy
 - MLP ranker can filter out a large amount of irrelevant documents
 - CNN is applied to a small set of candidates

DeText-CNN with Real-time Inference

- Compact CNN structure with small dimensions can perform well
- CNN computation time grows linearly with # of retrieved documents

Table 7: Number of CNN filters in DeText-CNN on NDCG@10.

#Filters	People Search	Job Search	Help Center
64	+3.02%	+4.65%	+11.56%
128	+3.07%	+4.81%	+11.94%
256	+3.10%	+4.82%	+12.37%
512	+3.16%	+4.92%	+12.74%

Online Deployment

- General Challenge
 - * computation grows linearly with # of documents
- Challenge for BERT serving
 - * 12-layer Google BERT has 110 million parameters
 - * Could be 30x times slower than CNN

LiBERT (LinkedIn pretrained BERT model)

- Fewer parameters -> smaller latency

	Google BERT	LiBERT_6_LAYER
Corpus	Wikipedia + BooksCorpus 3.3B words	LinkedIn search queries, member profiles, job docs, etc. 2.9B words
Layers	12	6
Hidden Layer Dimension	768	512
Total Params	110M	34M

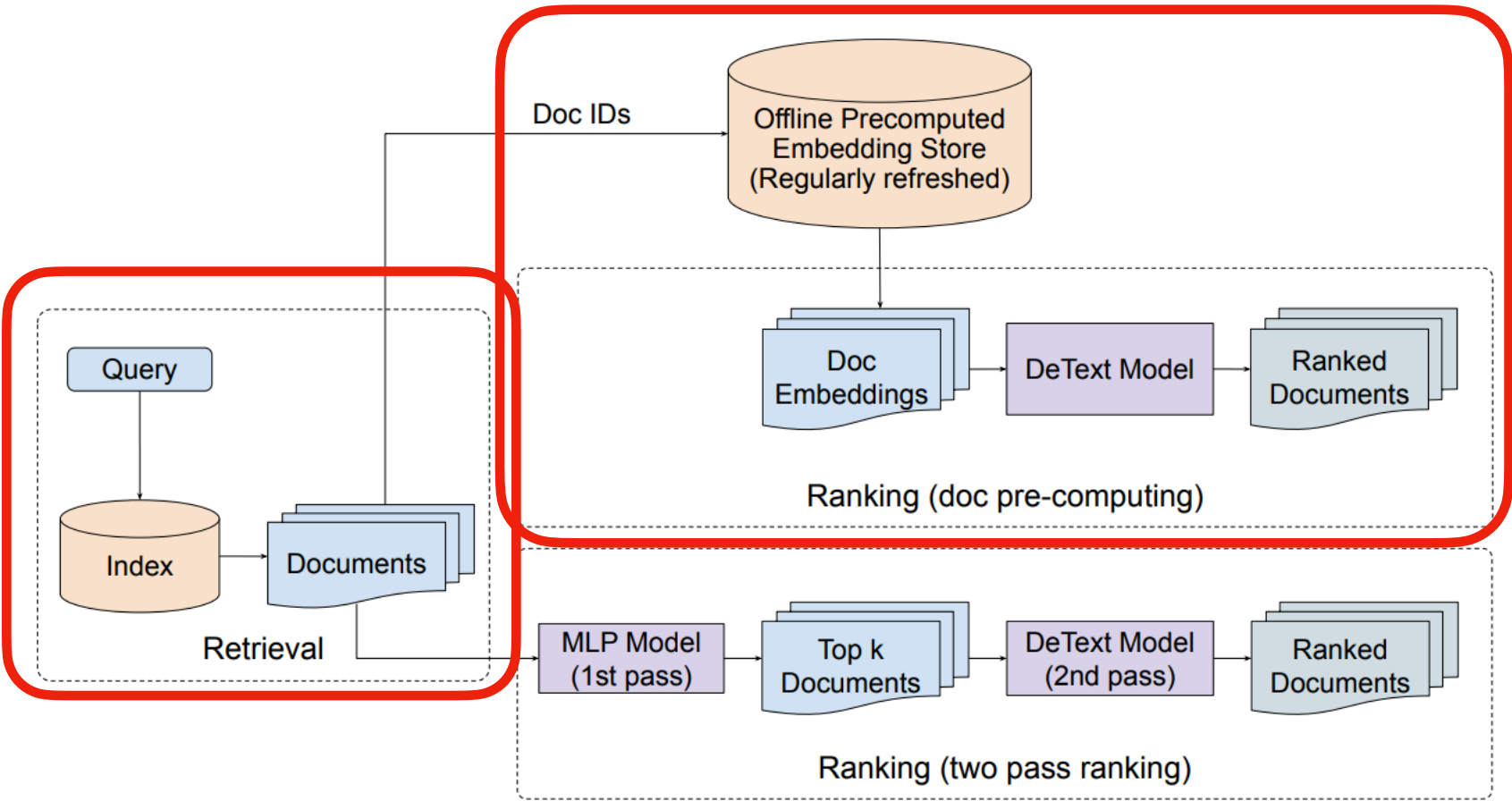
- Better relevance

Table 4: LinkedIn data for BERT pretraining.

Data Source	Description	# Words
Search queries	Query reformulation pairs	204M
Member profiles	Member headlines and summaries	98M
	Member position titles and descriptions	105M
Job posts	Job titles and descriptions	217M
Help center	Queries and doc titles	61M

Embedding Pre-computation

- DeText uses representation based model, adopt precomputing approach
- For Offline
 - document embeddings are pre-computed with BERT
 - Key-value store, key is document id, value is embedding vectors (daily)
- For Online
 - Fetch the document embeddings from pre-computed embedding store
 - Computation cost -> network communication cost



A close-up, low-angle shot of a chocolate cake with white frosting and a blue candle. The cake is in the foreground, and the background is a blurred city street at night with buildings and lights.

Introduction

Background

DeText Framework with BERT

Experiments

Application

Dataset

2개월간 트래픽에서 샘플링된 clickthrough data

- People Search
500만 쿼리
Document : member profiles (Headline, current position, pass position)
- Job search
150만 쿼리
Document : job post title, company name
- Help center
34만 쿼리
Document : title, example question

Metrics

Table 3: Online metrics definitions.

Metric	Definition
CTR@5	Proportion of searches that received a click at top 5 items
Session Success Rate	Proportion of search sessions that received a click. A new session is created if the user has no activity in help center search for 30 minutes.
Job Apply	Job search metric. Number of job applications from search.
Happy Path Rate	Help center search metric. Proportion of users who searched and clicked a document without using help center search again in that day, nor creating a ticket.

- Offline/online metrics 회사 기밀. 상대적인 값만 표기
- CTR@5
- 클릭받은 검색 세션 비율 (30분)
- 검색에서의 job application 수
- 하루에 문서를 재검색하지 않고 클릭한 사용자의 비율 (다시 검색 하지 않고)

Baseline Models

- XGBOOST - Hyper-parameters(pairwise vs list wise, number of tree) LinkedIn search engine에서 최적화 되어 있음
- Hand-crafted traditional features
 - Text matching features : cosine similarity, jacquard similarity , semantic matching features
 - Personalization features : social network distance between the searcher and profiles, searcher’s title overlapping with the job post title
 - Document popularity features : static rank of a member profile, clickthrough rate of a job

Search Ranking Experiments (Offline Experiments)

Overall

Table 5: Offline NDCG@10 score percentage lift in three searches over the production baseline XGBoost. † and ‡ denote statistically significant improvements at $p < 0.05$ using a two-tailed t-test over XGBoost and DeText-CNN, respectively.

Models	People Search	Job Search	Help Center
DeText-MLP	−0.07%	+0.05%	+0.15%
DeText-CNN	+3.02% [†]	+4.65% [†]	+11.56% [†]
DeText-LiBERT	+3.38% ^{†‡}	+6.14% ^{†‡}	+13.94% ^{†‡}

- DeText-MLP : DeText with only MLP and LTR layers on traditional features
- People Search, Job Search 도 성능 향상이 크지만, Help Center 가 가장 큰 개선이 됨
- Help Center Search : 유사한 시나리오의 문서가 많음, “how to hide my profile updates” vs “sharing profile changes with your networks”
- People Search : exact matching이 중요. “Twitter” vs “Facebook” , word embedding은 비슷 하지만 정확하게 결과를 리턴 해줘야
- Job Search : Help Center Search 과 People Search 중간

LiBERT vs BERT_{BASE}

Table 6: General BERT vs in-domain BERT on NDCG@10.

Model	People Search	Job Search	Help Center
DeText-CNN	+3.02%	+4.65%	+11.56%
DeText-BERT _{BASE}	+3.08%	+3.60%	+13.80%
DeText-LiBERT	+3.38%	+6.14%	+13.94%

- People Search, Job Search는 DeText-LiBERT가 상당히 개선됨
- Help Center는 구성된 어휘가 wikipedia와 가깝다. 비슷한 결과를 얻음
- 단, LiBERT가 BERT_{BASE} 보다 1/3 파라미터만 사용 함으로 가치가 있음

Search Ranking Experiments (Offline Experiments)

Text Embedding Interaction

Table 8: Text embedding interaction in DeText-CNN on NDCG@10.

Interaction	People	Job	Help Center
cosine	+2.67%	+4.25%	+11.02%
cosine, hadamard	+3.02%	+4.65%	+11.56%
cosine, concat	+2.39%	+4.62%	+10.09%
cosine, hadamard, concat	+2.84%	+4.73%	+11.49%

- Interaction method는 cosine + Hadamard (+ concat) 조합이 최상의 결과를 얻을 수 있음

Traditional Features

Table 9: Traditional features for DeText-CNN models on NDCG@10. The first row does not use any traditional features.

Trad-ft	Rescale	Norm	People	Job	Help Center
<i>X</i>	<i>X</i>	<i>X</i>	−4.52%	−9.98%	+11.07%
✓	<i>X</i>	<i>X</i>	+2.31%	+3.17%	+11.13%
✓	<i>X</i>	✓	+2.47%	+3.44%	+11.55%
✓	✓	<i>X</i>	+2.71%	+4.49%	+11.24%
✓	✓	✓	+3.02%	+4.65%	+11.56%

- Traditional features 를 사용 하는것이 중요
- Element-wise rescaling, normalization 이 도움이 됨

Search Ranking Experiments (Offline Experiments)

Multiple Fields

Table 10: The impact of using multiple fields. In the single target field setting, the most important field is used: headline for people search and job post title for job search. In this experiment, all the traditional features are excluded. Note that DeText-CNN with a single target field is a special version of CLSM model [26] that operates on words.

Model	#fields	People	Job
DeText-CNN (CLSM on words)	single	−5.20%	−12.83%
DeText-LiBERT	single	−3.14%	−10.30%
DeText-CNN	multiple	−4.52%	−9.98%
DeText-LiBERT	multiple	−2.51%	−7.20%

- 여러 필드를 사용 하는것이 중요
- Single field 에서는 가장 중요한 필드가 사용됨 (headline or title)
- Traditional features 는 제외

Search Ranking Experiments (Online Experiments)

- 2주 이상 20% 이하로 각 모델로 수행
- LiBERT 모델은 daily로 embedding을 새로 고침
- Job Search 같은 경우 새로운 job posting이 빈번 해서 DeText-LiBERT 를 feature work로 남겨둠
- People search, Help center search에서 DeText-LiBERT 가 일관되게 우수
- Importance of contextual embedding on capturing deep semantics between queries and documents

Table 11: Online experiments of DeText-CNN and DeText-LiBERT.

Search	Model	Metrics	Percentage Lift
People search	DeText-CNN	CTR@5	+1.13%
		Session Success Rate	neutral
	DeText-LiBERT	CTR@5	+1.56%
		Session Success Rate	+0.23%
Job search	DeText-CNN	CTR@5	+3.16%
		Job Apply	+0.73%
Help center	DeText-CNN	Happy Path Rate	+15.0%
		Session Success Rate	+6.1%
	DeText-LiBERT	Happy Path Rate	+26.1%
		Session Success Rate	+11.1%

Search Ranking Experiments (Latency Performance)

Table 12: The latency at 99 percentile on people search.

Model	Deployment Strategy	Time
DeText-CNN People	all-decoding	+55ms
DeText-CNN People	two pass ranking	+21ms
DeText-LiBERT people	doc pre-computing	+43ms
DeText-BERT _{BASE} people	doc pre-computing	+71ms

- Two pass ranking 가 latency 줄이는데 효과적임.
- A/B 테스트에서 all-decoding, two pass ranking의 관련성 차이는 없었음

- DeText-LiBERT 모델을 통해서 서비스 가능한 latency 확보
- DeText-BERT_{BASE} 보다 빠름

Time : additional latency compared to XGBoost baseline

A close-up, low-angle shot of a chocolate cake with white frosting and a blue candle. The cake is in the foreground, and the background is a blurred city street at night with warm lights and buildings.

Introduction

Background

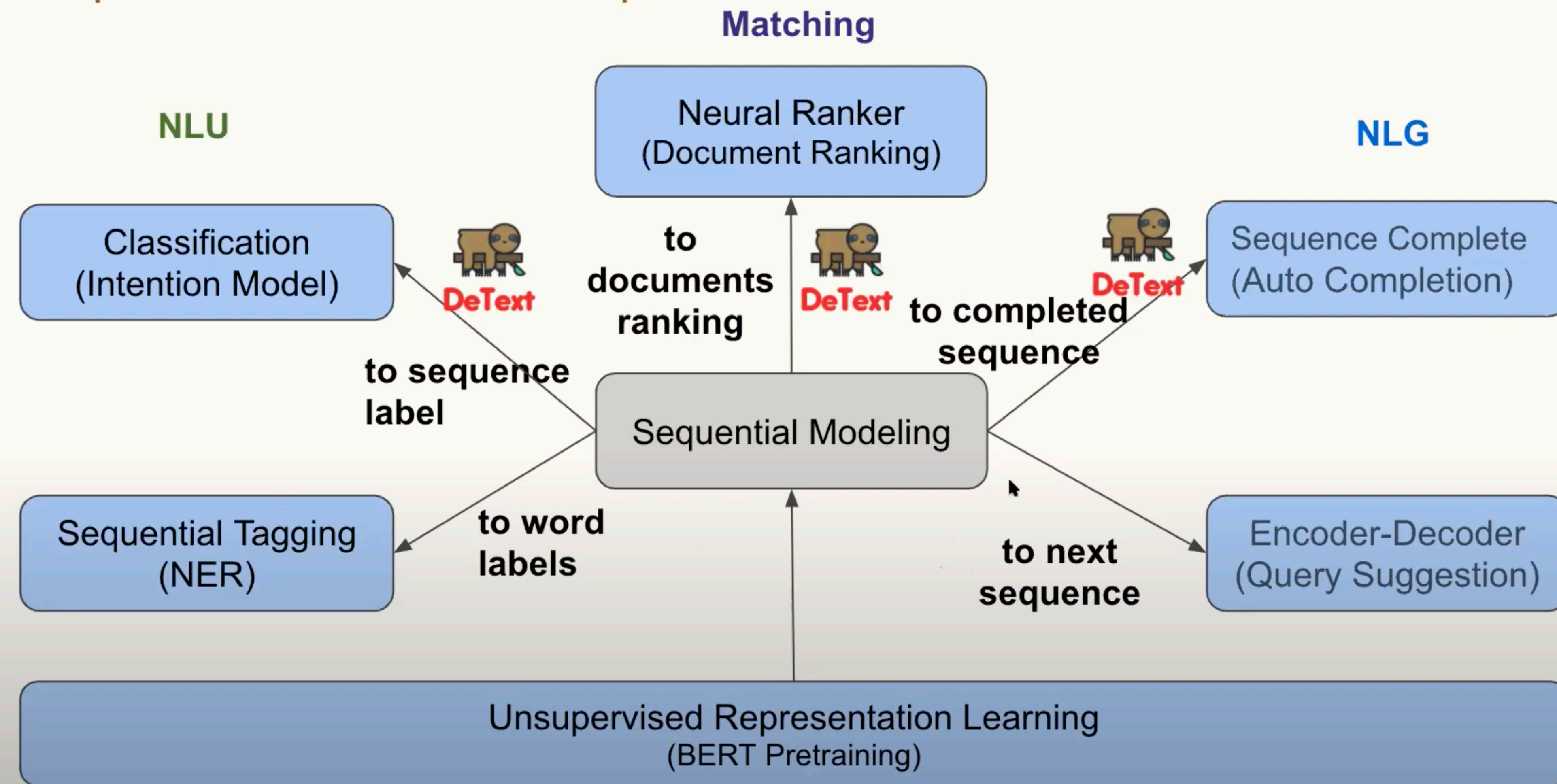
DeText Framework with BERT

Experiments

Application

Representative Deep NLP Tasks

Representative Deep NLP Tasks



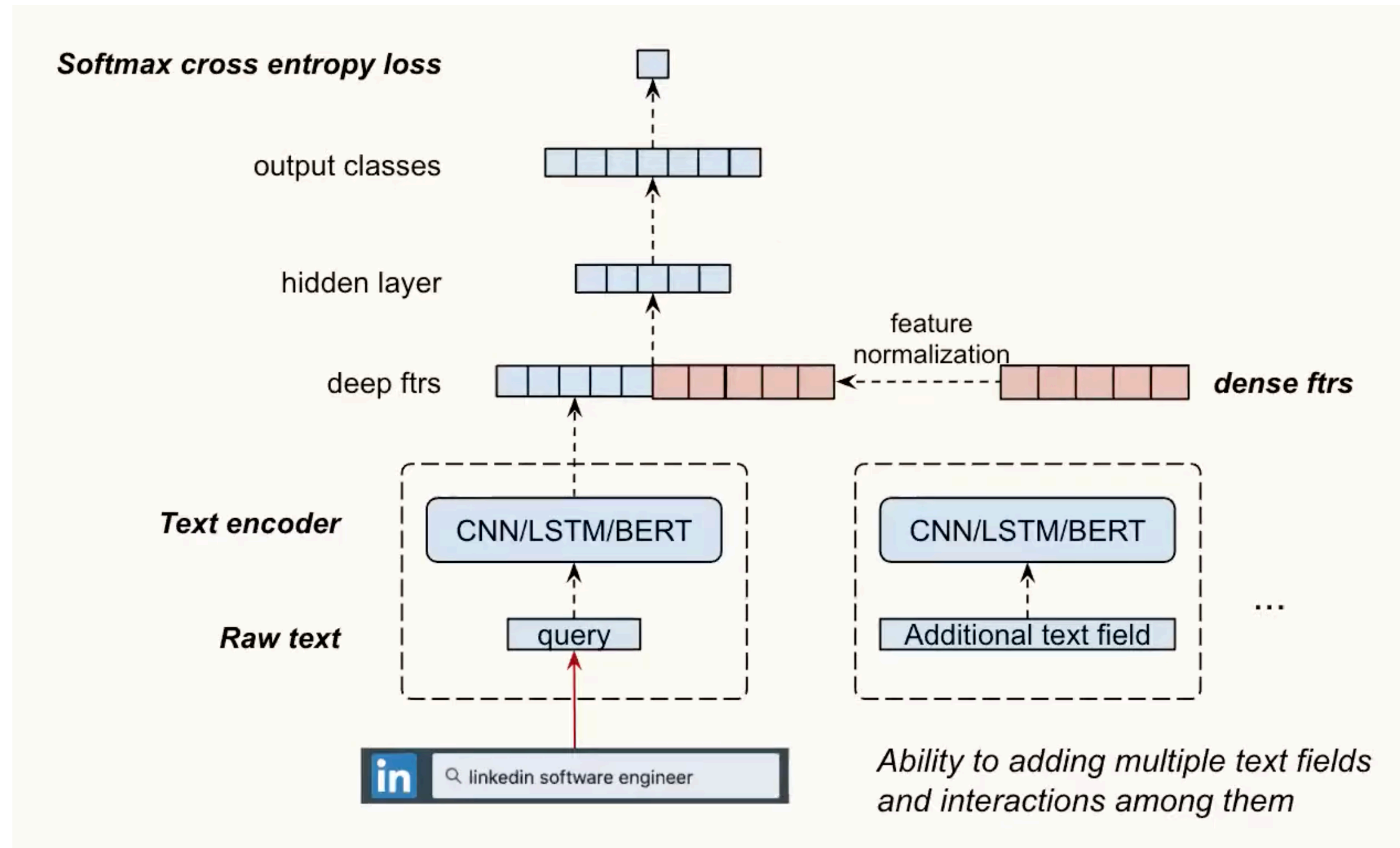
Example : Query Intent Classification

Input

- Query Text
- Dense Features

Predict : Search Intent

- People
- Job
- Content
- Company
- School
- Group
- Learning



Example : Job Recommendation, Query Auto Completion

Table 13: Offline experiments of DeText-CNN on job recommendation and query auto completion datasets. Both improvements are statistically significant at $p < 0.05$.

Tasks	Metrics	Percentage Lift
Job Recommendation	AUC	+3.01%
Query Auto Completion	MRR@10	+4.72%

Job Recommendation

- Input : (user id, job post id, applied for the job or no)
- Source Field : headline, job title, company, skill
- Target Field : including job title, job company, job skill, job country
- Baseline : logistic regression
- Pointwise 사용 (MLP의 hidden layer 없이 사용)
- Traditional features는 baseline 과 동일

Query Auto Completion

- Source Field : headline, job title, company title
- Target Field : completed query
- Baseline : XGBoost with traditional hand-crafted features
- listwise LTR 사용

Q&A

감사합니다.